1.5.2 Signal correlation

Different signal types are usually good for different tasks. Also, a single signal type is typically unable to answer all the questions you might have. We will discuss signal correlation in detail in chapter 11; however, I'd like to provide you with a quick motivational example here.

Let's say you are on call and get paged (in the middle of the night). You fire up your laptop and look at a Grafana dashboard linked in the alert you received. From the dashboard, which, say, plots Prometheus metrics, you can gather that certain services in your microservices app have an increased error rate in the past 20 minutes. You have a hunch that this may be related to the recent upgrade of a service. You use the trace ID from an exemplar (http://mng.bz/D4Jw) embedded in the Prometheus metrics as an entry point to have a closer look at the involved services, using the distributed tracing tool Jaeger. In Jaeger, you see the traces duration distribution and error codes and drill down into the logs of the service to verify your hypothesis.

In this scenario, you correlated the different signal types (metric to traces to logs) and were able to confirm your hypothesis quickly and effectively. A simple form of correlation is a time-based one, which is something you get in many frontends, such as Grafana, without additional effort out of the box. This means you use a time period to query different backends to manually find signals that tell you the whole story.

1.5.3 Portability

Avoid lock-in by choosing open standards and open source. This ensures you can use the same mechanisms and tooling in different environments (e.g., cloud providers or on-premises infrastructure) without having to change much of your setup. For example, if you instrument your applications using OpenTelemetry, then you can switch from one destination to another without having to change your application or agents.

Summary

- Cloud-native systems, such as Kubernetes, produce a large volume of telemetry signals.
- Sources of telemetry signals can be your code, APIs (e.g., an AWS service), a database, and so on.
- The term *telemetry* refers to the process of collecting signals from sources, processing them via agents, and ingesting them into observability destinations, where they can be consumed by humans and machines.
- In general, *observability* means getting actionable insights from signals to understand and influence the system under observation.
- There are many use cases for observability, from reducing MTTR to cost optimizations to increasing developer productivity.
- Observability is relevant to different roles (developers, DevOps, nontechnical roles, etc.) to perform their typical tasks.